LAB 6: SPI

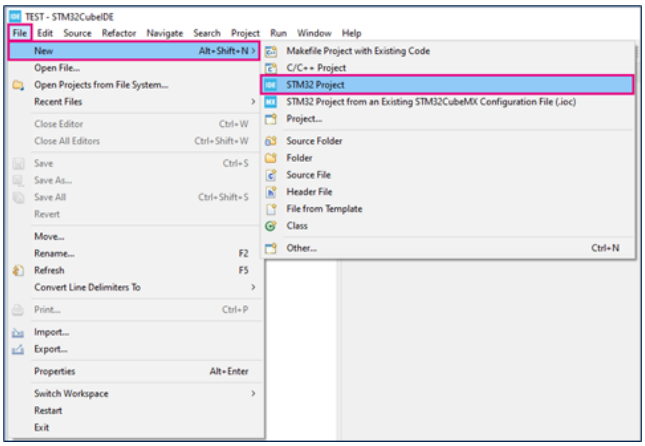
Part 1: Connect 2 boards for communication

1.1. Objectives

- Learn how to connect 2 boards using SPI in STM32CubeMX.
- Display the output on LCD
- Generate code in STM32CubeMX and using HAL functions.

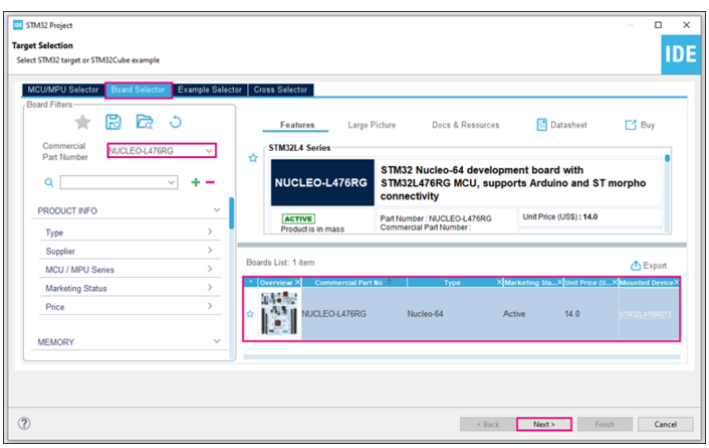
1.2. Create the project in STM32CubeIDE

- **File** > **New** > **STM32 Project** in main panel.



This example uses the NUCLEO-L476RG board.

- Select **NUCLEO-L476RG** board using **Board selector** as shown in the figure below:

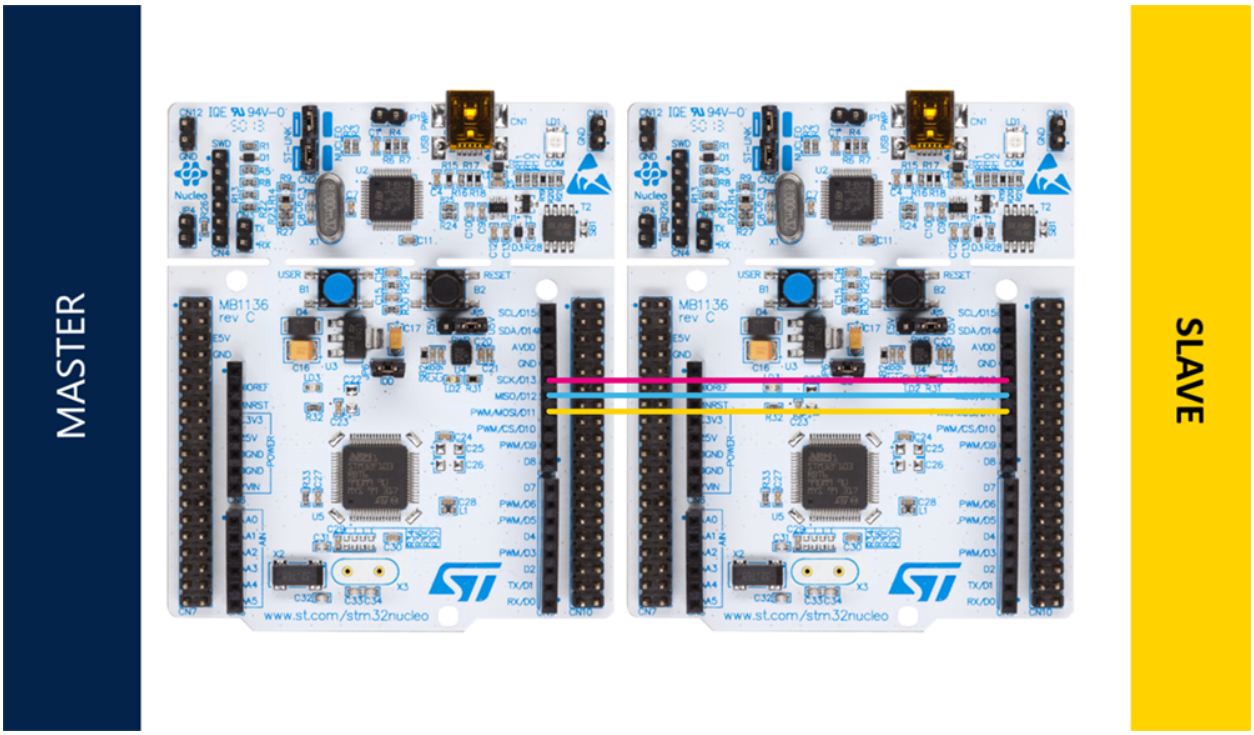


- Save the project:

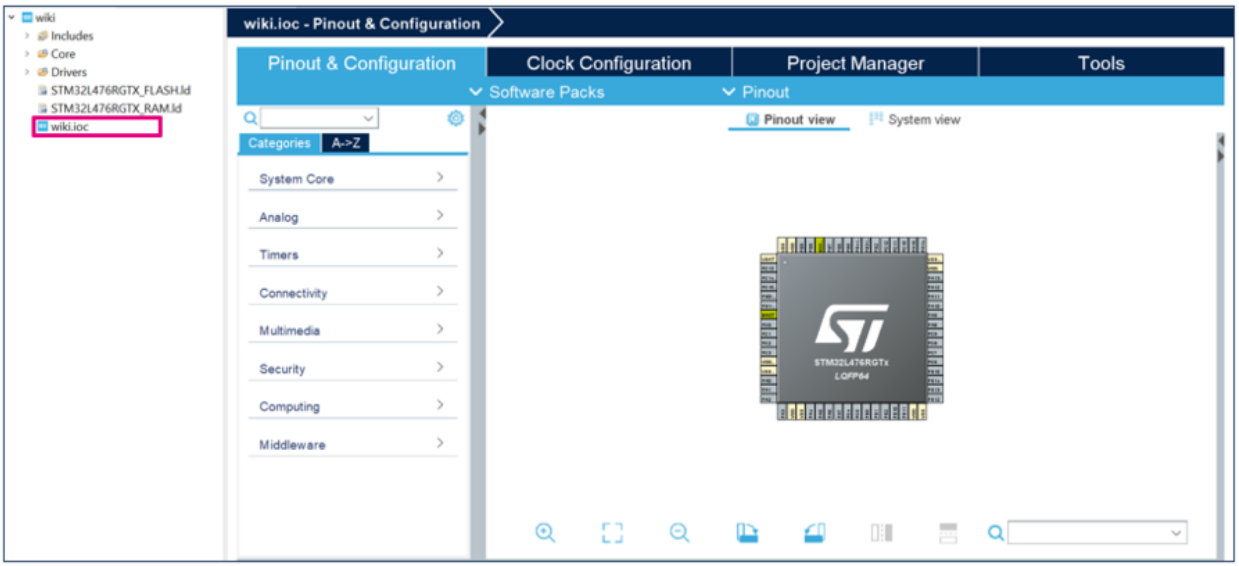
1.3. Configure Master

The following table shows the connection between two NUCLEO-L474RG boards using the SPI bus, requiring only three wires

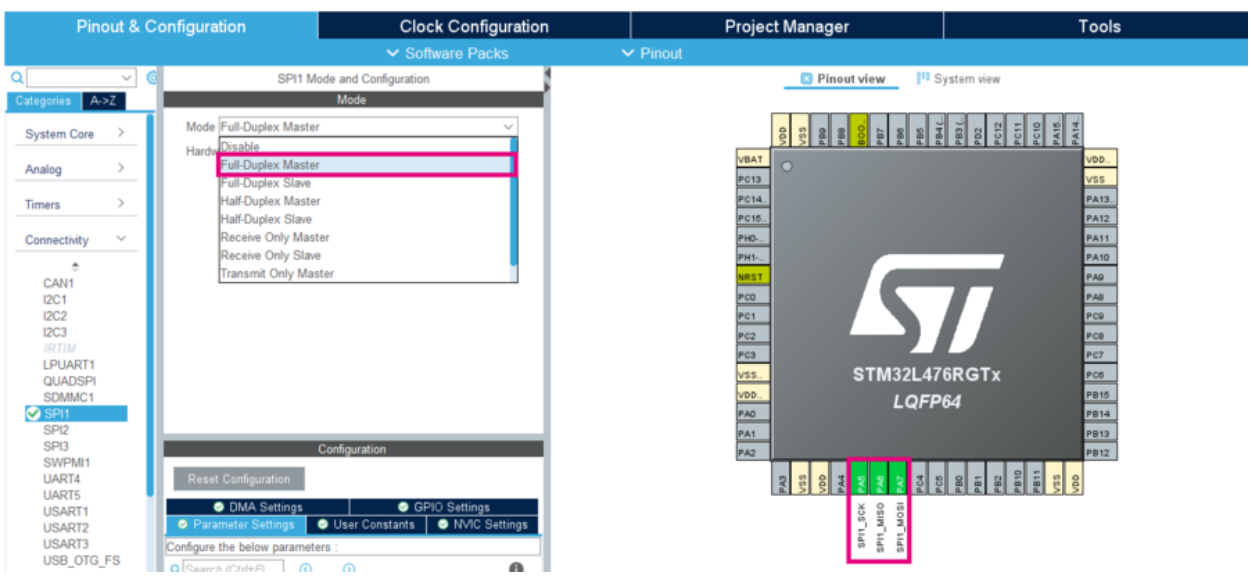
NUCLEO-L476RG MASTER	NUCLEO-L476RG SLAVE	Pin description
PA5	PA5	SCK
PA6	PA6	MISO
PA7	PA7	MOSI

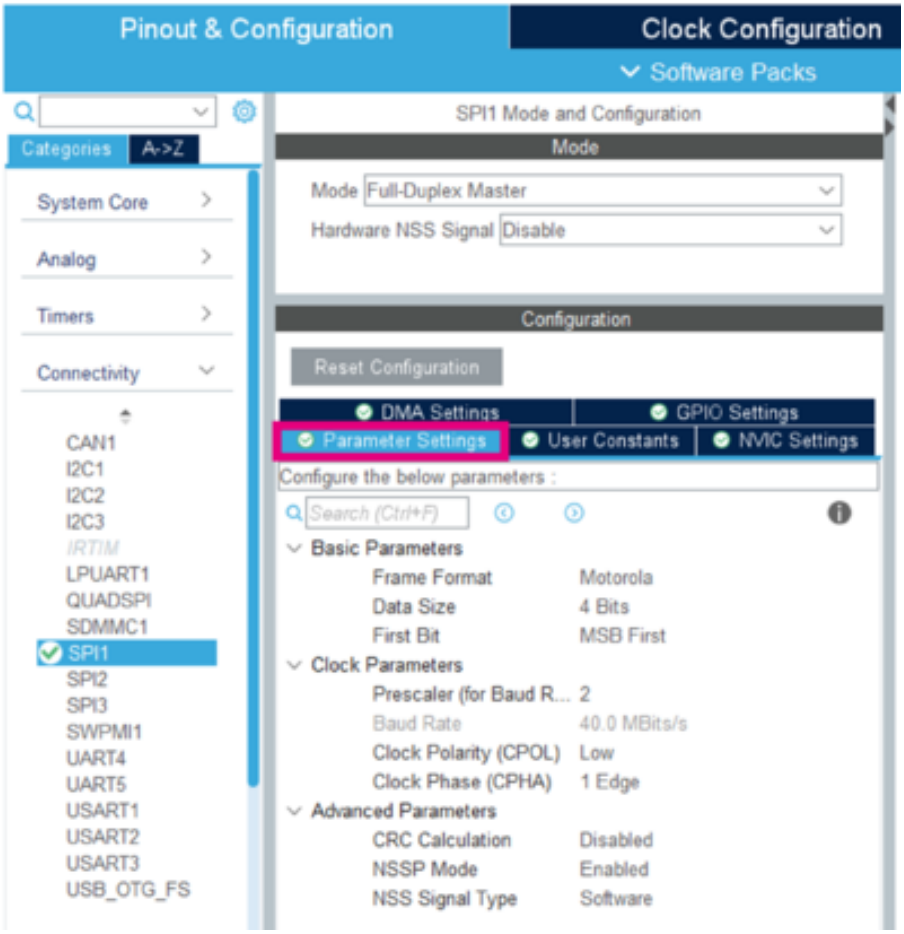


- Configure SPI on master board
  - ► Open the STM32CubeMX project from the workspace.

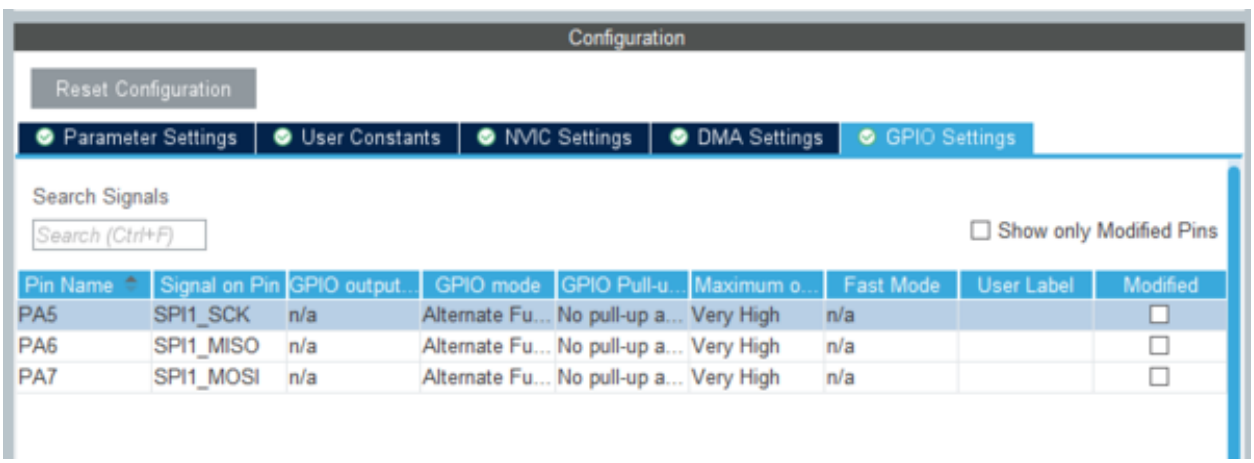


- Configure SPI on master board
  - Under Connectivity, select SPI1.
  - Select Full-Duplex Master mode:
    - Pin PA5 as SCK
    - Pin PA6 as MISO
    - Pin PA7 as MOSI

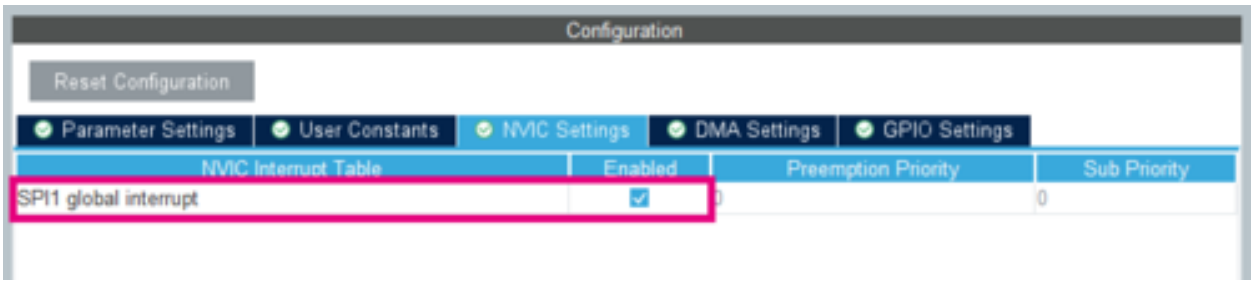




- **GPIO settings**
  - ► Leave the GPIO settings in the default configuration, as shown below.

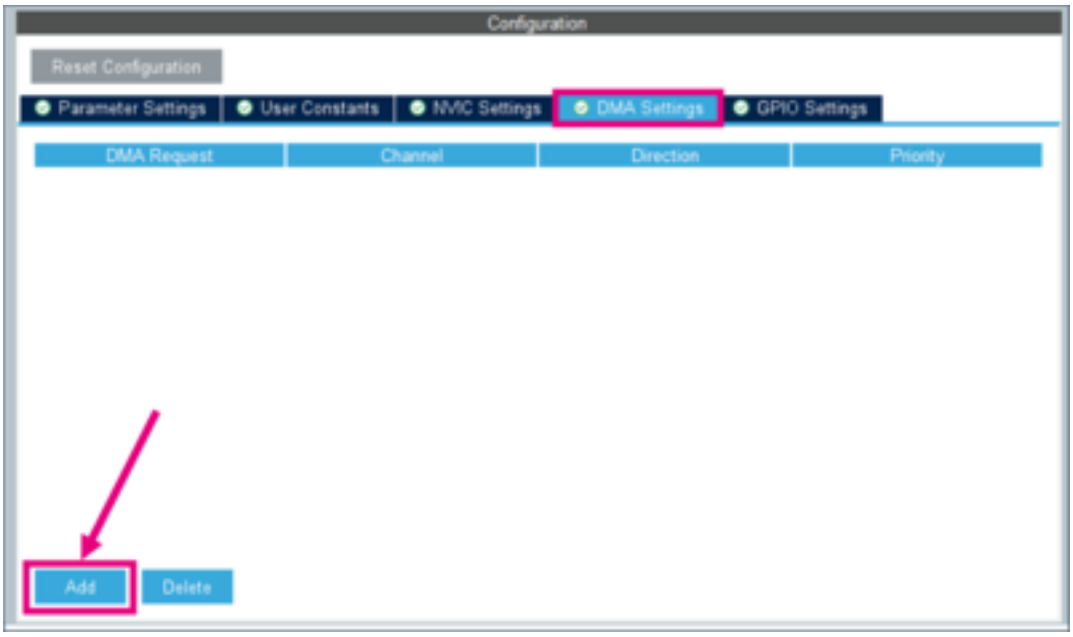


- NVIC and DMA settings
  - ► Interrupt mode only
    - ■ Go to the NVIC settings tab
    - ■ Enable SPI1 global interrupt

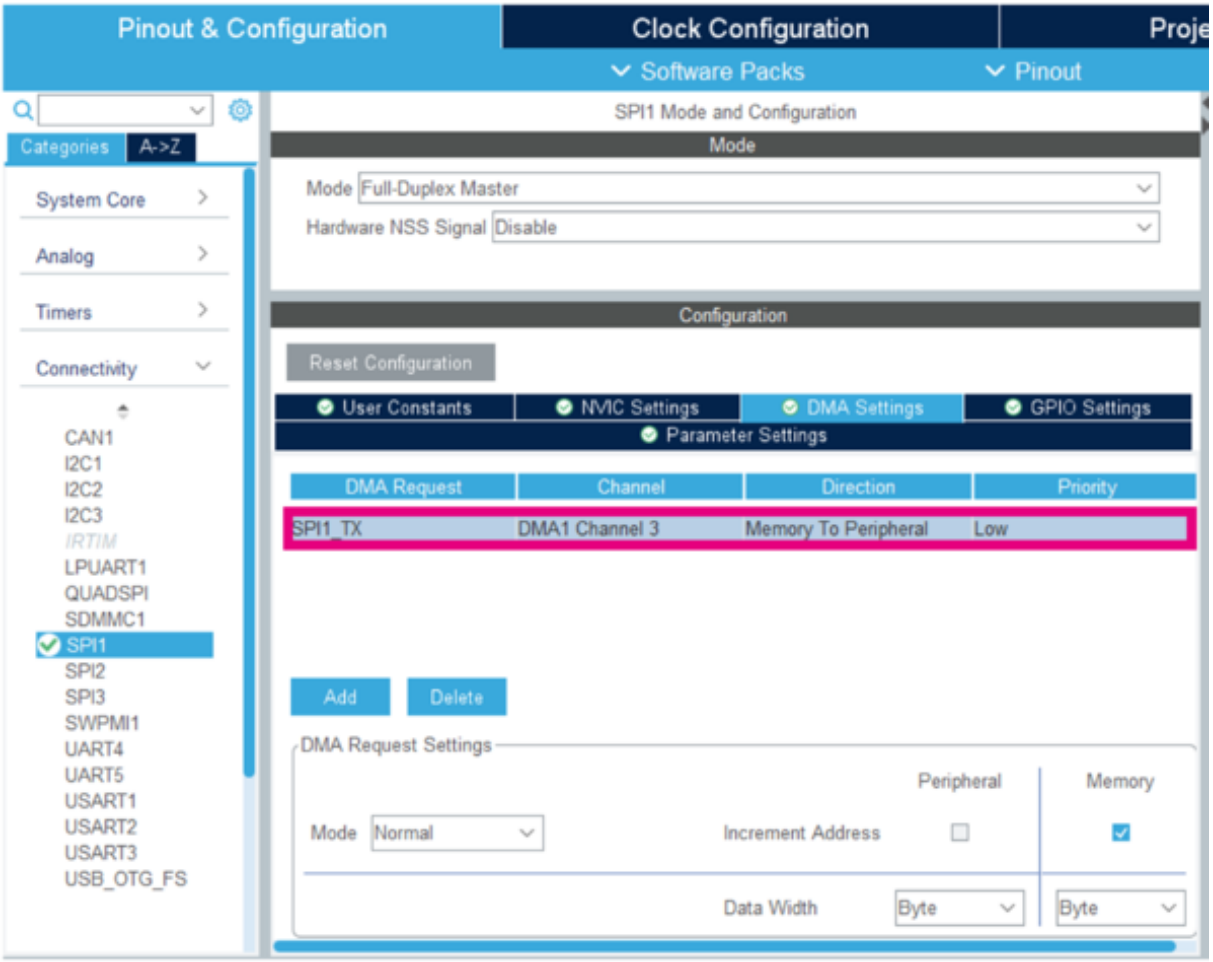


- NVIC and DMA settings
  - ► DMA mode only
    - Go to the DMA settings tab and click the Add button.

■ Go to the DMA settings tab and click the Add button.



- NVIC and DMA settings
  - DMA mode only
    - Add SPI1_TX for the master board:



- Generate source code and edit main.c

Information

Insert your code between the ***/* USER CODE BEGIN PV */*** and ***/* USER CODE END PV */*** tags.

```
/* USER CODE BEGIN PV */
/* Private variables */
uint8_t TX_Buffer [] = "A" ; // DATA to send
/* USER CODE END PV */
```

Information

Insert your code between the ***/* USER CODE BEGIN 2 */*** and ***/* USER CODE END 2 */*** tags.

- **Blocking mode**

```
/* USER CODE BEGIN 2 */
  HAL_SPI_Transmit(&hspi1, TX_Buffer, 1, 1000); //Sending in Blocking mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

- **Interrupt mode**

```
/* USER CODE BEGIN 2 */
  HAL_SPI_Transmit_IT(&hspi1, TX_Buffer, 1); //Sending in Interrupt mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

- **DMA mode**

```
/* USER CODE BEGIN 2 */
  HAL_SPI_Transmit_DMA(&hspi1, TX_Buffer, 1);; //Sending in DMA mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

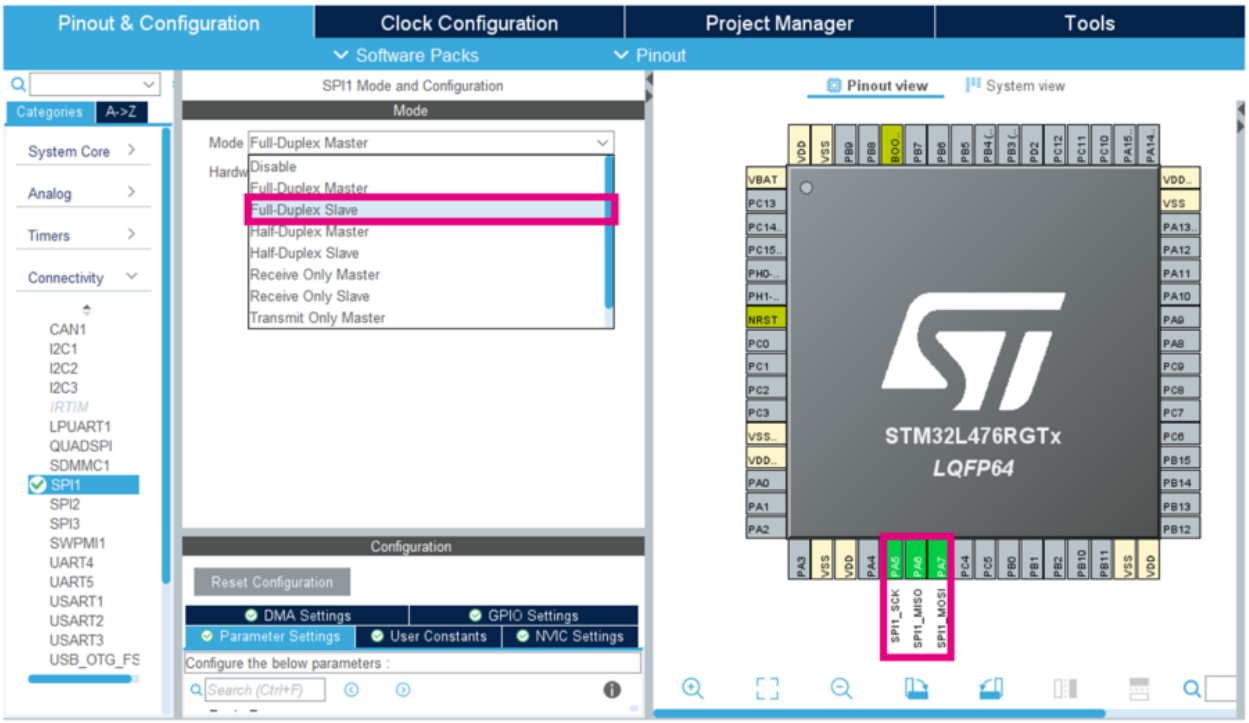
- Create a new project, as described in #Create master project in STM32CubeIDE above
  - Configure SPI on slave board
  - SPI settings
    - Under Connectivity, select SPI1.
    - Select Full-Duplex Slave mode:
      - Pin PA5 as SCK
      - Pin PA6 as MISO
      - Pin PA7 as MOSI
    - Leave the SPI parameters settings in their default configuration, as shown below:

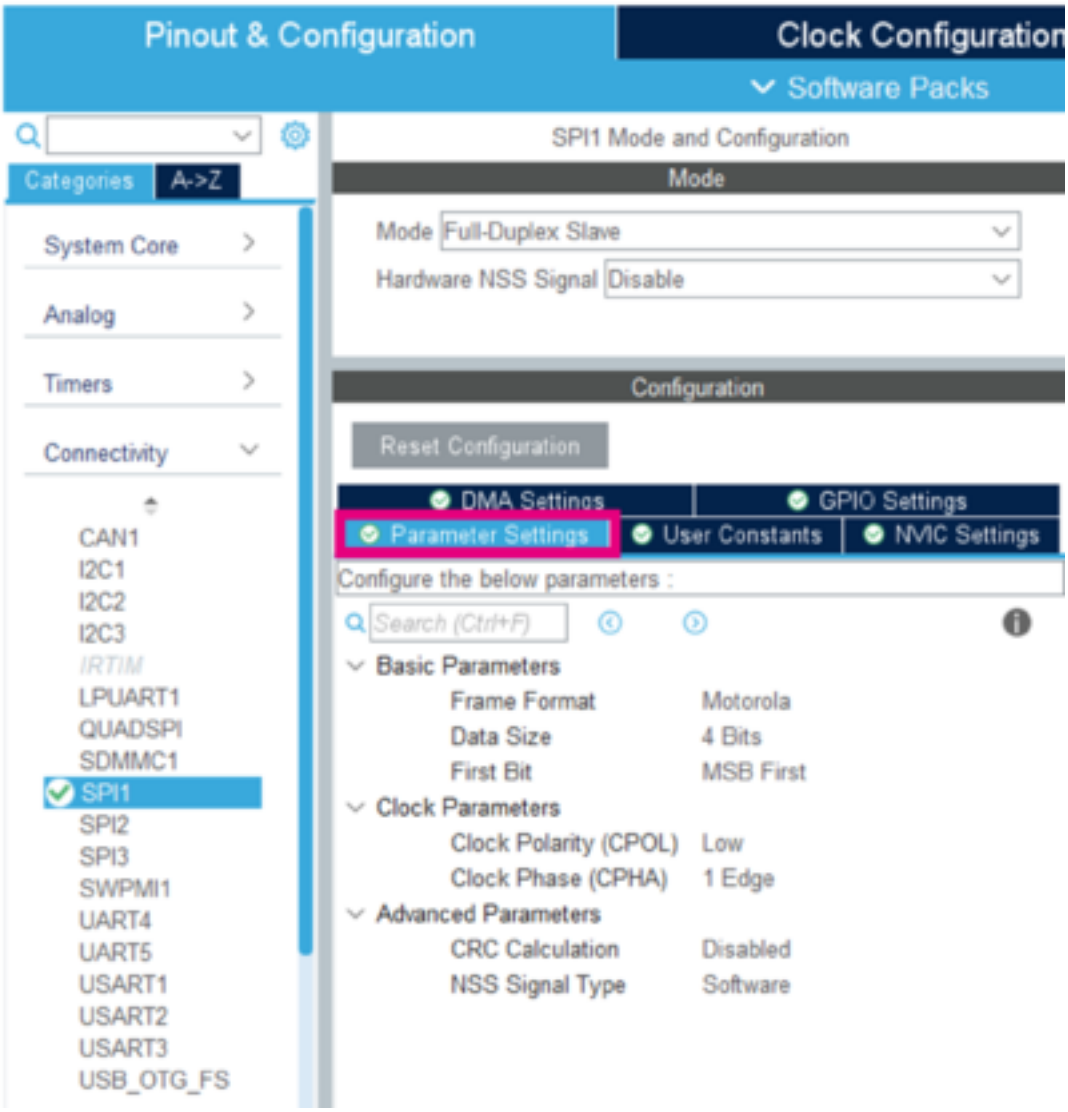
1.4 Compile and flash

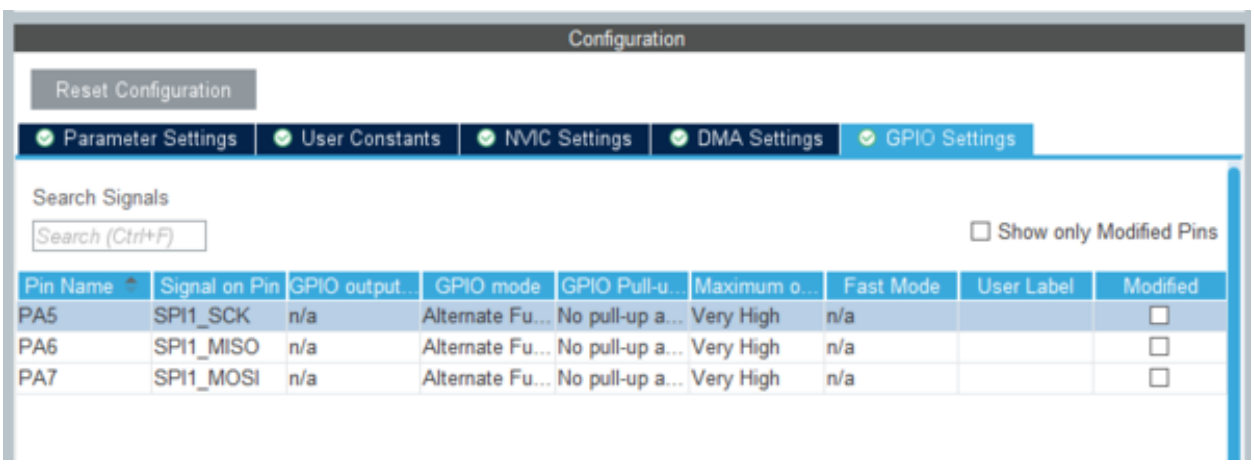
- Click the **Build** button
- Click the **Run** button to run it

1.5. Configure Slave

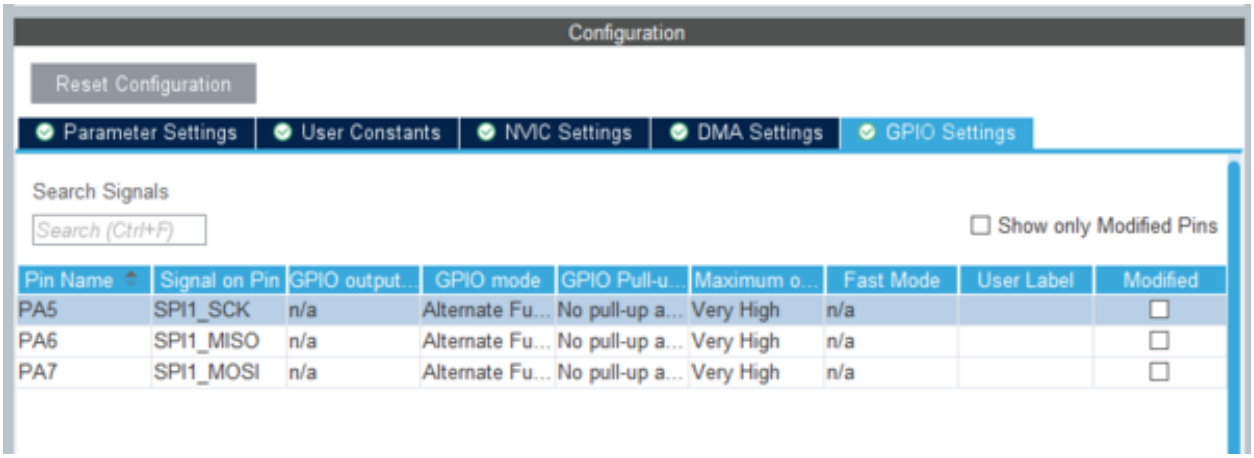
- Create a new project, as described in #Create master project in STM32CubeIDE above
  - Configure SPI on slave board
  - SPI settings
    - Under Connectivity, select SPI1.
    - Select Full-Duplex Slave mode:
      - Pin PA5 as SCK
      - Pin PA6 as MISO
      - Pin PA7 as MOSI
    - Leave the SPI parameters settings in their default configuration, as shown below:



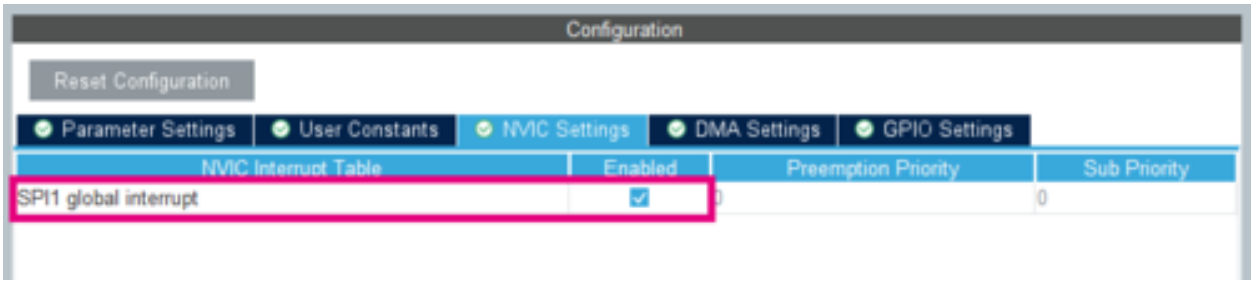




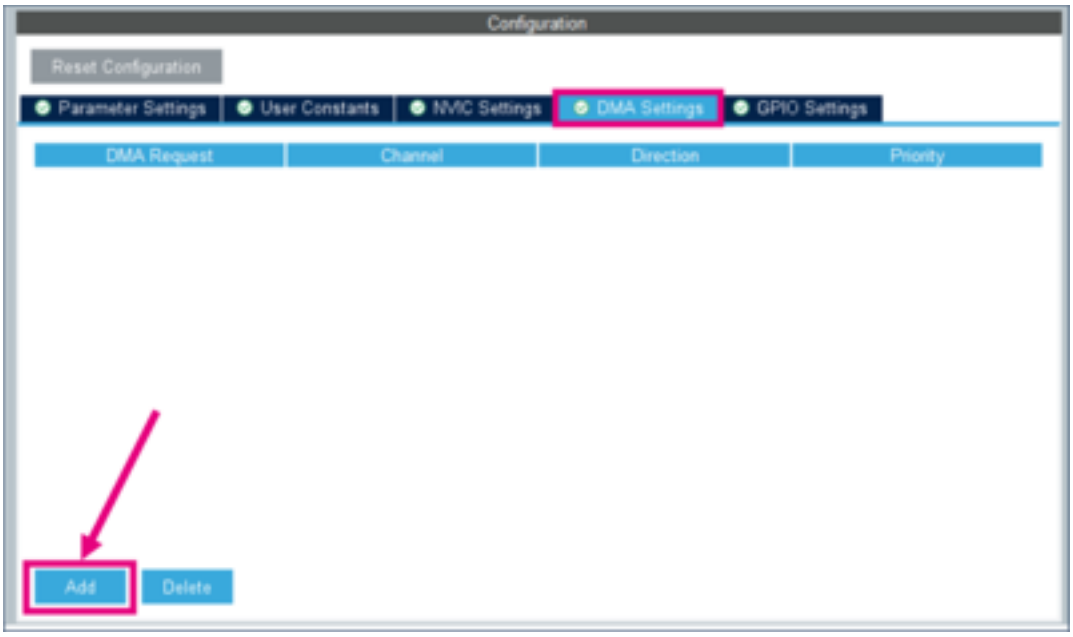
- **GPIO settings**
  - ► Leave the GPIO settings in the default configuration, as shown below.



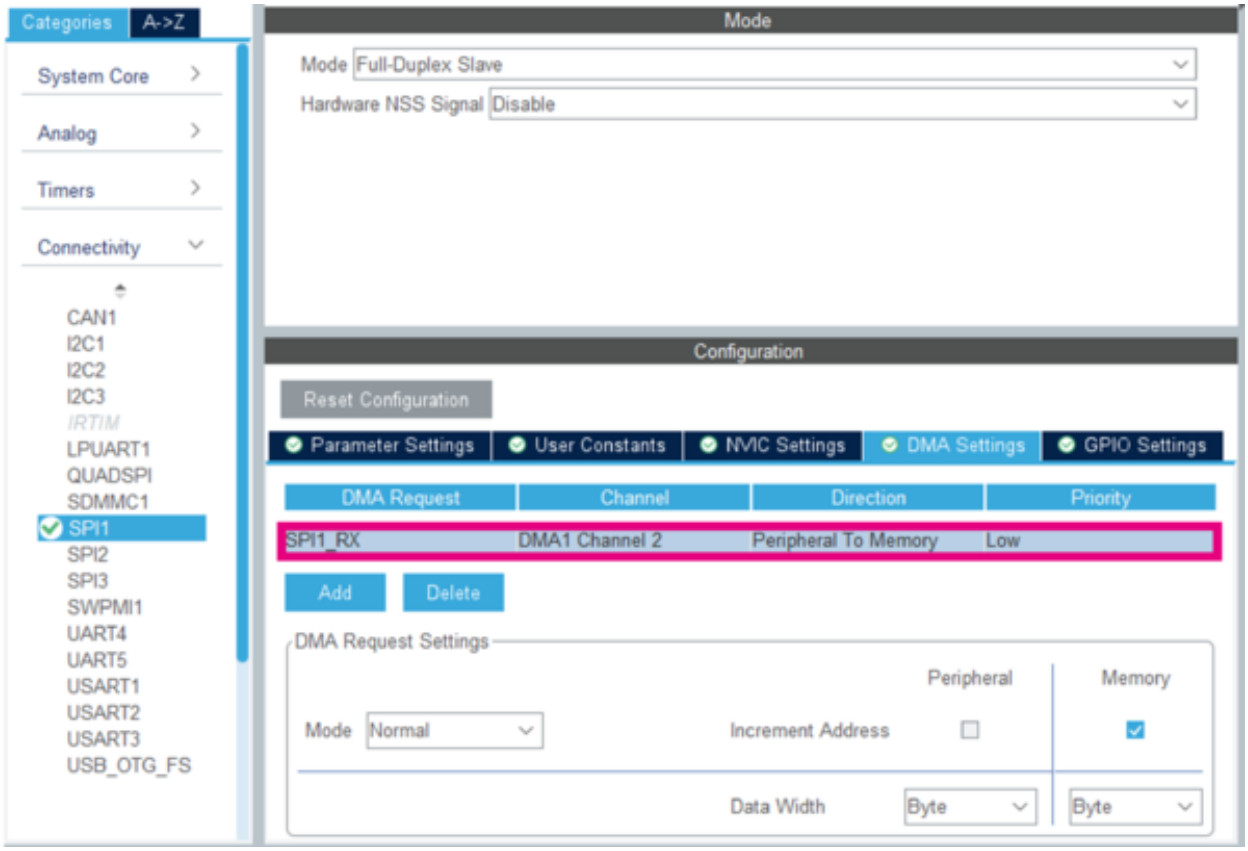
- NVIC and DMA settings
  - Interrupt mode only
    - Go to the NVIC settings tab
    - Enable SPI1 global interrupt



- NVIC and DMA settings
  - DMA mode only
    - Go to the DMA settings tab and click the Add button.



- NVIC and DMA settings
  - DMA mode only
    - Add SPI1_RX for the master board:



- Generate source code and edit main.c

Information

Insert your code between the ***/* USER CODE BEGIN PV */*** and ***/* USER CODE END PV */*** tags.

```
/* USER CODE BEGIN PV */
/* Private variables */
uint8_t RX_Buffer [1] ; // DATA to receive
/* USER CODE END PV */
```

Information

Insert your code between the ***/* USER CODE BEGIN 2 */*** and ***/* USER CODE END 2 */*** tags.

- **Blocking mode**

```
/* USER CODE BEGIN 2 */
  HAL_SPI_Receive(&hspi1, RX_Buffer, 1, 1000); //Receiving in Blocking mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

- **Interrupt mode**

```
/* USER CODE BEGIN 2 */
  HAL_SPI_Receive_IT(&hspi1, RX_Buffer, 1); //Receiving in Interrupt mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

- **DMA mode**

```
/* USER CODE BEGIN 2 */
  HAL_SPI_Receive_DMA(&hspi1, RX_Buffer, 1); //Receiving in DMA mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

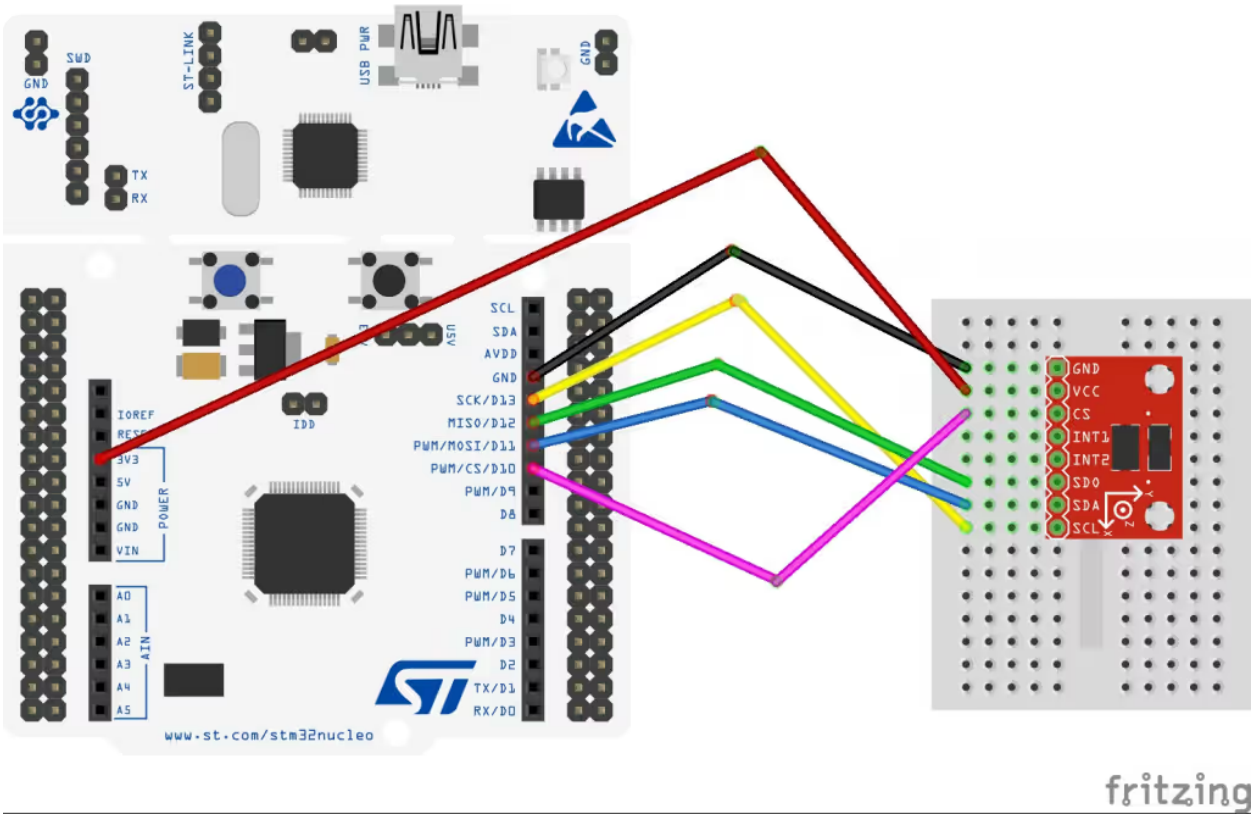
1.5 Compile and flash

- Click the **Build** button
- Click the **Run** button to run it

Part 2:

Objectives

- Use the ADXL345 with the STM32l476RG
- Print the output on the LCD.



The sensor is powered with 3.3V from the Nucleo board itself. The pin connections are as follows:

Nucleo Pin	Sensor Pin	Description
SCK (Clock)	**SCL (Clock)**	Provides the clock signal for SPI communication.
MISO	**SDO (Serial Data Out)**	Transfers data from the sensor to the Nucleo.
MOSI	**SDA (Serial Data)**	Transfers data from the Nucleo to the sensor. In SPI mode, this SDA pin acts as **SDI (Serial Data Input)**.

Nucleo Pin	Sensor Pin	Description
CS	CS	Chip Select signal to enable or disable the sensor during communication.

12	SDO/ALT ADDRESS	Serial Data Output (SPI 4-Wire)/Alternate I²C Address Select (I²C).
13	SDA/SDI/SDIO	Serial Data (I²C)/Serial Data Input (SPI 4-Wire)/Serial Data Input and Output (SPI 3-Wire).
14	SCL/SCLK	Serial Communications Clock. SCL is the clock for I²C, and SCLK is the clock for SPI.

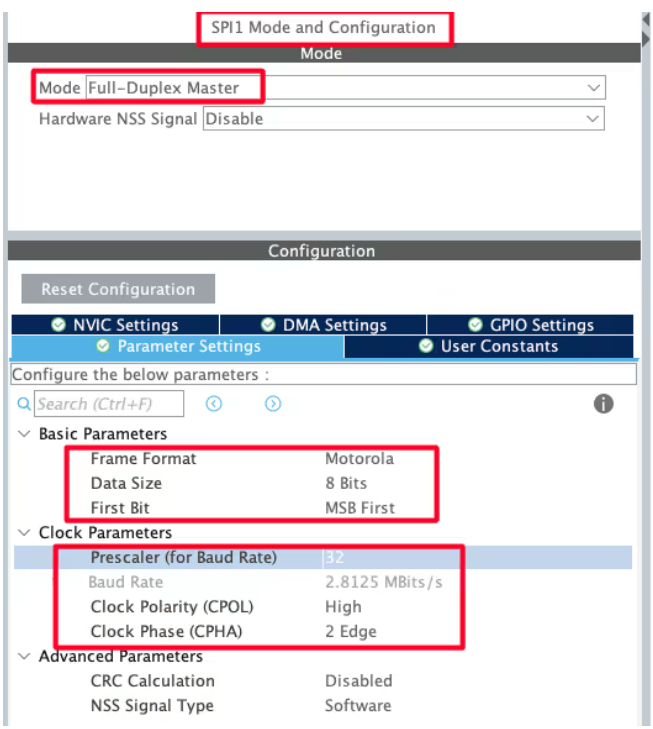
Clock Configuration

Use the PLL to run the system at maximum 80MHz.

SPI Configuration

As per the ADXL345 datasheet, the maximum SPI clock can be set to 5MHz, so we will keep our SPI clock below this value. Also the sensor follows the SPI MODE 3, CPOL=1, CPHA=1.

mode. The maximum SPI clock speed is 5 MHz with 100 pF maximum loading, and the timing scheme follows clock polarity (CPOL) = 1 and clock phase (CPHA) = 1. If power is applied



The SPI is configured in Full Duplex mode, so the STM32 as master can send and receive data at the same time. The Data size is set to 8 Bits because the ADXL345 only supports 8 bit data transfer. The data will be arranged in MSB first format.

I have used the Prescaler of 16 to reduce the SPI clock to 2.8MB/s. This is to make sure that our SPI clock remain lower than 5MB/s (MAX for ADXL345). Also note that the CPOL is HIGH (1) and CPHA is set to 2 edge (CPHA=1).

STM32 SPI Code Example – Read & Write Functions Using HAL

ADXL Write Function

Unlike I2C, the SPI does not have different slave addresses to identify the read and write operations. Therefore the master itself needs to inform the slave whether it wants to perform a write operation or a read operation. Along with that the master also need to inform whether it is performing a multibyte read or write.

Below is the image showing the write operation on the ADXL345 using SPI.

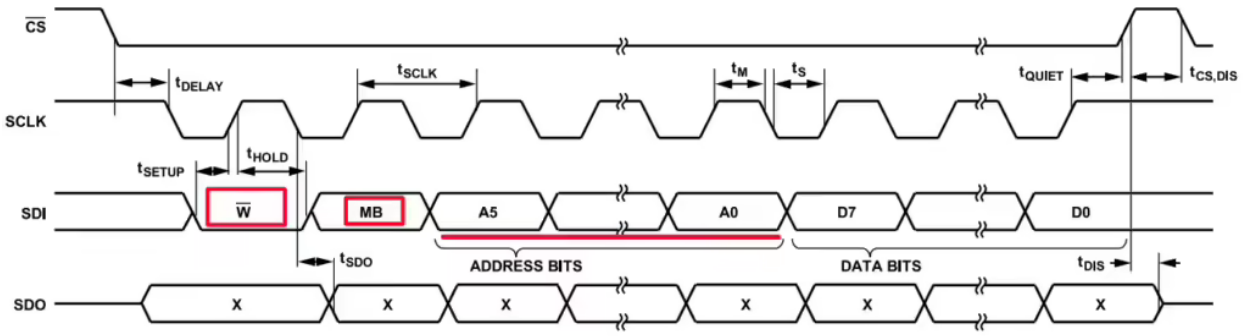


Figure 37. SPI 4-Wire Write

As you can see in the image above, there are 2 additional bits attached with the Address bits. The Address is only 6bit long and then we have the Multibyte (MB) bit and Read Write (R/W) bit. The R/W bit must be set to 0 for the write operation. Also the MB bit should be set to 1 for writing multiple bytes to the device.

Below is the function to write the data to the slave.

```
void adxl_write (uint8_t Reg, uint8_t data)
{
	uint8_t writeBuf[2];
	writeBuf[0] = Reg|0x40;  // multibyte write enabled
	writeBuf[1] = data;
	HAL_GPIO_WritePin (GPIOB, GPIO_PIN_6, GPIO_PIN_RESET); // pull the cs pin low to enable the slave
	HAL_SPI_Transmit (&hspi1, writeBuf, 2, 100);  // transmit the address and data
	HAL_GPIO_WritePin (GPIOB, GPIO_PIN_6, GPIO_PIN_SET); // pull the cs pin high to disable the slave
}
```

The parameter of the function adxl_write are:

- @Reg is the register address inside the slave device, where the master wants to write the data to.
- @data is the data, the master wants to store in the above address.

Since we are writing 2 bytes, register address and data, we need to store both the bytes in an array. Then set the R/W bit to 0 for write operation and MB bit to 1 for multibyte operation. Our final data for the first byte transfer will be b01xxxxxx. Here the 'x' represents the 5 bit register address.

To send the data via the SPI, we need to follow the procedure as mentioned below.

- Pull the CS low to select the slave.
- Call the function *HAL_SPI_Transmit* to transmit the 2 byte array we just defined. The timeout for the transfer operation is set to 100ms.
- Pull the CS pin high to disable the slave.

ADXL Read Function

Just like write operation, the master needs to inform the slave that this is a read operation. To do so, the R/W but must be set to 1, indicating the Read command.

Below is the image showing the write operation on the ADXL345 using SPI.

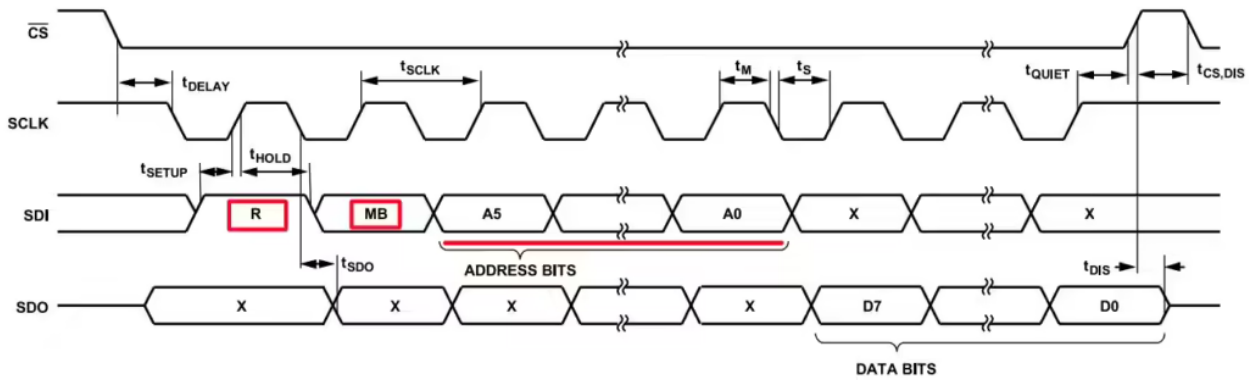


Figure 38. SPI 4-Wire Read

As you can see in the image above, there are 2 additional bits attached with the Address bits. The Address is only 6bit long and then we have the Multibyte (MB) bit and Read Write (R/W) bit. The R/W bit must be set to 1 for the read operation. Also the MB bit should be set to 1 for reading multiple bytes from the device.

Below is the function to read the data from the slave.

```
void adxl_read (uint8_t Reg, uint8_t *Buffer, size_t len)
{
	Reg |= 0x80;  // read operation
	Reg |= 0x40;  // multibyte read
	HAL_GPIO_WritePin (GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);  // pull the cs pin low to enable the slave
	HAL_SPI_Transmit (&hspi1, &Reg, 1, 100);  // send the address from where you want to read data
	HAL_SPI_Receive (&hspi1, Buffer, len, 100);  // read 6 BYTES of data
	HAL_GPIO_WritePin (GPIOB, GPIO_PIN_6, GPIO_PIN_SET);  // pull the cs pin high to disable the slave
}
```

The parameter of the function adxl_write are:

- @Reg is the register address inside the slave device, where the master wants to read the data from.
- @Buffer is the pointer to the buffer where the the received data will be stored.
- @len is the number of bytes, the master wants to read.

The R/W bit must be set to 1 for read operation and MB bit to 1 for multibyte operation. Our final data for the Register Address will be b11xxxxxx. Here the 'x' represents the 5 bit register address.

To read the data via the SPI, we need to follow the procedure as mentioned below.

- Pull the CS low to select the slave.
- Call the function *HAL_SPI_Transmit* to transmit the modified register address, where we want to read the data from. The timeout for the transfer operation is set to 100ms.
- Call the function HAL_SPI_Receive to receive the data from the slave device. The 6 bytes of data will be stored in the Buffer array.
- Pull the CS pin high to disable the slave.

ADXL Initialization

We will first check if the slave device is responding, by reading the ID Register (0x00). The ADXL should respond with the value 0xE5.

Register 0x00—DEVID (Read Only)

Table 20. Register 0x00

D7	D6	D5	D4	D3	D2	D1	D0
1	1	1	0	0	1	0	1

The DEVID register holds a fixed device ID code of 0xE5 (345 octal).

```
void adxl_init (void)
{
	uint8_t chipID=0;
	adxl_read(0x00, &chipID, 1);
```

If the DEV_ID returned is 0xE5, we will proceed with the initialisation.

Now we will modify POWER_CTL Register (0x2D) and DATA_FORMAT Register (0x31). First RESET all bits of POWER_CTL register by writing 0 to them.

```
        if (chipID == 0xE5)
        {
                adxl_write (0x2d, 0x00);  // reset all bits; standby
                adxl_write (0x2d, 0x08);  // measure=1 and wake up 8hz
```

Set the MEASURE bit, RESET the SLEEP bit and SET the frequency in the WAKE UP bits

Register 0x2D—POWER_CTL (Read/Write)

D7	D6	D5	D4	D3	D2	D1	D0
0	0	Link	AUTO_SLEEP	Measure	Sleep	Wakeup	

Next, in the DATA_FORMAT Register, Set the RANGE using D0 and D1.

Register 0x31—DATA_FORMAT (Read/Write)

D7	D6	D5	D4	D3	D2	D1	D0
SELF_TEST	SPI	INT_INVERT	0	FULL_RES	Justify	Range	

Wakeup Bits

These bits control the frequency of readings in sleep mode as described in Table 27.

Table 27. Frequency of Readings in Sleep Mode

Setting		
D1	D0	Frequency (Hz)
0	0	8
0	1	4
1	0	2
1	1	1

```
                adxl_write (0x31, 0x01);  // 10bit data, range= +- 4g
        }
}
```

The main function

Inside the main function, we will first initialise the ADXL.

```
int main()
```

```
{
 ....
 adxl_init();  // initialize adxl
```

We will write the rest of the code in the while loop.

```
 while (1)
 {
          adxl_read (0x32, RxData, 6);
```

Here we will first read 6 bytes starting from the Register 0x32. The data is stored in the Registers 0x32 to 0x37 in the form of DATA X0, DATA X1, DATA Y0, DATA Y1, DATA Z0, DATA Z1.

Register 0x32 to Register 0x37—DATAX0, DATAX1, DATAY0, DATAY1, DATAZ0, DATAZ1 (Read Only)

These six bytes (Register 0x32 to Register 0x37) are eight bits each and hold the output data for each axis. Register 0x32 and Register 0x33 hold the output data for the x-axis, Register 0x34 and Register 0x35 hold the output data for the y-axis, and Register 0x36 and Register 0x37 hold the output data for the z-axis. The output data is twos complement, with DATAx0 as the least significant byte and DATAx1 as the most significant byte, where x represent X, Y, or Z. The DATA_FORMAT register (Address 0x31) controls the format of the data. It is recommended that a multiple-byte read of all registers be performed to prevent a change in data between reads of sequential registers.

Now we need to combine the DATA X0, DATA X1 into single 10 bit value and this can be done by

```
int16_t x = ((RxData[1]<<8)|RxData[0]);
int16_t y = ((RxData[3]<<8)|RxData[2]);
```

```
int16_t z = ((RxData[5]<<8)|RxData[4]);
```

Next we will convert this data into the g form in order to check for the acceleration in specific axis. As you can check above in the initialisation part, we have set the range of ±4 g. According to the datasheet, for the range of ±4 g, the sensitivity is 128LSB/g.

SENSITIVITY	Each axis				
Sensitivity at X_{OUT}, Y_{OUT}, Z_{OUT}	All *g*-ranges, full resolution	230	256	282	LSB/*g*
	±2 *g*, 10-bit resolution	230	256	282	LSB/*g*
	±4 *g*, 10-bit resolution	115	128	141	LSB/*g*
	±8 *g*, 10-bit resolution	57	64	71	LSB/*g*
	±16 *g*, 10-bit resolution	29	32	35	LSB/*g*

So to convert into g, we need to divide the value by 128.

```
    xg = (float)x/128;
    yg = (float)y/128;
    zg = (float)z/128;

    HAL_Delay(1000);
  }
}
```

Deliverables

- Report
- Video of the 2 parts
- Project compressed file with the code for each of the 2 parts